

# MARAN PMS — Prompt 45: Recepción — Reservas, Check-in/out, Clientes, Tarifas

## Archivos principales

- `client/src/pages/new-reservation.tsx` — `NewReservationPage`
  - `client/src/pages/check-in.tsx` — `CheckInPage`
  - `client/src/pages/check-out.tsx` — `CheckOutPage`
  - `client/src/pages/guests.tsx` — `GuestsPage` + `GuestFormDialog`
  - `client/src/pages/rate-plans.tsx` — `RatePlansPage` + `RatePlanFormDialog`
  - `client/src/pages/planning.tsx` — para el fix de apertura desde celda
  - `server/routes.ts` — endpoints check-in/check-out
  - `server/storage.ts` — `searchGuests`, `updateGuest`, `updateRatePlan`
- 

## SECCIÓN 1 — CREACIÓN DE RESERVA

### 1.1 Buscador de huésped: búsqueda por apellido, nombre, DNI, email o teléfono

**Problema:** buscar "lisman" no encuentra a "Luciana Lisman" porque el buscador no matchea apellido solo.

**Fix backend** — `server/storage.ts`, función `searchGuests`:

```
// ESTADO ACTUAL (ya busca por firstName, lastName, email, documentNumber)
// Verificar que también incluya phone y que el OR funcione correctamente
async searchGuests(query: string): Promise<Guest[]> {
  return db.select().from(guests).where(
    or(
      ilike(guests.firstName, `${query}%`),
      ilike(guests.lastName, `${query}%`),
      ilike(guests.email, `${query}%`),
```

```

        ilike(guests.documentNumber, `%${query}%`),
        ilike(guests.phone, `%${query}%`),          // ← agregar si falta
    )
    ).limit(20);
}

```

**Fix frontend** — en `GuestSelector` (componente reutilizable usado en `new-reservation` y `check-in`):

El buscador debe disparar la búsqueda con el string completo tal como se escribe. Si actualmente divide por palabras o solo busca `firstName`, corregirlo para que envíe la query completa a `GET /api/guests/search?q=lisman` y muestre resultados que contengan el término en cualquier campo.

En los resultados del dropdown mostrar: `{firstName} {lastName} — DNI {documentNumber}` para que el operador confirme que es la persona correcta.

## 1.2 Empresa asociada al cliente — autocompletado

**Objetivo:** si el huésped seleccionado tiene `companyId` en su perfil, cargar automáticamente la empresa en el campo `CompanySelector` al crear una reserva.

**Fix en** `NewReservationPage` ( `client/src/pages/new-reservation.tsx` ):

```

// Cuando se selecciona un huésped, verificar si tiene companyId
const handleGuestSelect = (guest: Guest) => {
    setSelectedGuest(guest);
    if (guest.companyId) {
        // Buscar la empresa en la lista ya cargada
        const company = companies?.find(c => c.id === guest.companyId);
        if (company) setSelectedCompany(company);
    }
};

```

Asegurarse de que `companies` esté disponible en el componente (probablemente ya se usa para `CompanySelector`). Si no está, agregar:

```

const { data: companies } = useQuery<Company[]>({ queryKey: ["/api/companies"] })

```

El mismo fix aplicar en `CheckInPage` en el tab "Walk-in" donde también se selecciona huésped.

**Fix en** `GuestFormDialog` ( `client/src/pages/guests.tsx` ): agregar campo "Empresa" en

el formulario de edición/creación de huésped para que se pueda asignar `companyId` desde el perfil del huésped.

```
// Agregar al formData inicial:
companyId: guest?.companyId || "",

// Agregar campo en el formulario:
<div className="grid gap-2">
  <Label>Empresa asociada</Label>
  <Select
    value={formData.companyId || ""}
    onChange={(v) => setFormData({ ...formData, companyId: v || null })}
  >
    <SelectTrigger><SelectValue placeholder="Sin empresa" /></SelectTrigger>
    <SelectContent>
      <SelectItem value="">Sin empresa</SelectItem>
      {companies?.map(c => (
        <SelectItem key={c.id} value={c.id}>{c.razonSocial}</SelectItem>
      ))}
    </SelectContent>
  </Select>
</div>
```

---

### 1.3 Apertura desde planning: fijar habitación y categoría

**Objetivo:** cuando se hace clic en una celda del planning para crear reserva, la categoría y número de habitación deben llegar preseleccionados y NO ser editables.

**Fix en** `planning.tsx` : al navegar a `/new-reservation` , pasar los parámetros como query string:

```
// Al hacer clic en celda del planning:
setLocation(`/new-reservation?roomId=${room.id}&roomTypeId=${room.roomTypeId}&dat
```

**Fix en** `NewReservationPage` : leer los parámetros de la URL al montar:

```
const [searchParams] = useSearchParams(); // o useLocation + parse manual

const prefilledRoomId = searchParams.get("roomId") || "";
const prefilledRoomTypeId = searchParams.get("roomTypeId") || "";
const prefilledDate = searchParams.get("date") || "";
const isFromPlanning = !!prefilledRoomId;
```

```
// Inicializar estado con los valores prefijados:
const [selectedRoomId, setSelectedRoomId] = useState(prefilledRoomId);
const [selectedRoomTypeId, setSelectedRoomTypeId] = useState(prefilledRoomTypeId)
const [checkInDate, setCheckInDate] = useState(prefilledDate || today);
```

**Cuando** `isFromPlanning === true`:

- El Select de "Tipo de Habitación" debe mostrarse como texto (no editable): `<span>{roomType?.name}</span>`
- El Select de "Habitación" debe mostrarse como texto: `<span>Hab.{room?.roomNumber}</span>`
- Ambos campos deben tener un badge o nota: "Preseleccionado desde el planning"

## 1.4 Tipo de camaje editable en nueva reserva y edición

**Diagnóstico:** en `NewReservationPage` el campo `bedTypeId` usa `useState<number | null>` pero el schema define `bedTypeId` como `varchar/UUID`. Esto causa que el Select no funcione correctamente.

**Fix:**

```
// ANTES:
const [bedTypeId, setBedTypeId] = useState<number | null>(null);

// DESPUÉS:
const [bedTypeId, setBedTypeId] = useState<string>("");

// Y en el Select:
<Select
  value={bedTypeId}
  onValueChange={setBedTypeId}
>
  {bedTypes?.filter(bt => bt.isActive).map(bt => (
    <SelectItem key={bt.id} value={bt.id}>{bt.name}</SelectItem>
  ))}
</Select>

// Al enviar:
bedTypeId: bedTypeId || null,
```

Aplicar el mismo fix en el formulario de **edición de reserva** (`ReservationDetailPage` o donde esté el dialog de edición de reserva existente).

---

## SECCIÓN 2 — CHECK-IN / CHECK-OUT

### 2.1 Check-in retroactivo con motivo obligatorio

**Problema actual:** el endpoint `POST /api/reservations/:id/check-in` rechaza check-ins con `diffDays < -1` (más de 1 día en el pasado).

**Cambio en backend** ( `server/routes.ts` ):

```
// Lógica actual (simplificada):
if (diffDays > 1 || diffDays < -1) {
  return res.status(400).json({ error: "..." });
}

// Nueva lógica:
if (diffDays > 0) {
  // Fecha futura: siempre bloquear
  return res.status(400).json({
    error: `No se puede hacer check-in en fecha futura. La reserva es para el ${c
  });
}

if (diffDays < -1) {
  // Fecha pasada (más de 1 día): permitir solo si viene motivo
  const { motivo } = req.body;
  if (!motivo || motivo.trim() === "") {
    return res.status(400).json({
      error: "CHECK_IN_RETROACTIVO",
      message: `La fecha de check-in es ${checkInDate}. Para registrar con fecha
      requiresMotivo: true
    });
  }
  // Registrar en changelog
  await db.insert(reservationChangelog).values({
    reservationId: req.params.id,
    fecha: new Date(),
    operador: (req as any).user?.username || "sistema",
    tipo: "checkin_retroactivo",
    descripcion: `Check-in retroactivo registrado el ${today} para fecha ${checkI
  });
}
```

**Cambio en frontend** ( `client/src/pages/check-in.tsx` ):

Cuando la mutación de check-in recibe el error `CHECK_IN_RETROACTIVO`, abrir un dialog de confirmación con:

- Mensaje explicando que la fecha de check-in es pasada
- Campo de texto obligatorio: "Motivo del check-in retroactivo"
- Botón "Confirmar igual" que reenvía la mutación con `{ motivo: texto }`
- Botón "Cancelar"

```
const [retroactivoDialog, setRetroactivoDialog] = useState(false);
const [retroactivoMotivo, setRetroactivoMotivo] = useState("");
const [pendingCheckInId, setPendingCheckInId] = useState<string | null>(null);

// En onError de checkInMutation:
onError: (error: any) => {
  if (error?.data?.error === "CHECK_IN_RETROACTIVO") {
    setPendingCheckInId(reservationId);
    setRetroactivoDialog(true);
    return;
  }
  // resto del manejo de error...
}

// Al confirmar:
const confirmRetroactivo = () => {
  if (!retroactivoMotivo.trim()) return;
  checkInMutation.mutate({
    id: pendingCheckInId!,
    body: { motivo: retroactivoMotivo }
  });
  setRetroactivoDialog(false);
};
```

---

## 2.2 Bloquear check-in en fechas futuras

Ya cubierto en 2.1 — el backend rechaza con mensaje claro cuando `diffDays > 0`. El frontend debe mostrar el mensaje de error del toast sin ofrecer opción de override.

---

## 2.3 Cerrar habitaciones desde el registro de salidas

**Problema:** en `CheckOutPage`, las reservas con `checkOutDate < today` (vencidas/no

cerradas) se muestran pero no se puede hacer check-out desde esa sección.

**Fix en** `CheckOutPage` : en la sección de reservas vencidas ( `overdueReservations` ), agregar botón "Cerrar ahora" que inicia el wizard de check-out normal:

```
{overdueReservations.map(res => (  
  <div key={res.id} className="flex items-center justify-between p-3 border round  
    <div>  
      <p className="font-medium">{res.guest?.firstName} {res.guest?.lastName}</p>  
      <p className="text-sm text-muted-foreground">  
        Hab. {res.room?.roomNumber} – Salida: {formatDateAR(res.checkOutDate)}  
      </p>  
    </div>  
    <Button  
      size="sm"  
      variant="outline"  
      onClick={() => startCheckout(res)}  
      data-testid={`button-checkout-overdue-${res.id}`}  
    >  
      Cerrar ahora  
    </Button>  
  </div>  
))}
```

El wizard de check-out ya existente maneja esta situación mediante `forceCheckout` .

Verificar que el backend permita check-out de reservas con `checkOutDate` anterior a hoy (el `diffDays < -1` en check-out backend debería ser más permisivo que el de check-in, o eliminarse para check-out de vencidas).

---

## 2.4 Resumen completo de cuenta antes de finalizar check-out

**Diagnóstico:** el wizard de check-out ya tiene un step 1 con "Resumen" que muestra cargos. Verificar que muestre:

- Total de cargos (alojamiento + extras)
- Total de pagos ya registrados
- Saldo pendiente (o cero si está saldado)
- Métodos de pago usados

Si el step 1 ya muestra esto correctamente, no hay cambio. Si el balance no se calcula correctamente, verificar que `folio?.balance` use la fórmula:

```
balance = totalRoomAmount + totalCharges - totalPayments
```

Y que se muestre con color rojo cuando `balance > 0` y verde cuando `balance <= 0`.

---

## SECCIÓN 3 — CLIENTES

### 3.1 Edición de huésped no borra datos existentes

**Diagnóstico:** `GuestFormDialog` inicializa `formData` con los campos del huésped, pero **solo incluye algunos campos** (`firstName`, `lastName`, `email`, `phone`, `documentType`, `documentNumber`, `nationality`, `localidad`, `vehículo`). Campos como `direccion`, `codigoPostal`, `fechaNacimiento`, `sexo`, `segment`, `cuilCuit`, `agencyId` **no están en el** `useState` inicial, por lo que al hacer `PATCH` se envían como `undefined` o vacíos, sobrescribiendo los valores existentes.

**Fix:** inicializar `formData` con TODOS los campos del schema de guest:

```
const [formData, setFormData] = useState<Partial<InsertGuest>>({
  firstName: guest?.firstName || "",
  lastName: guest?.lastName || "",
  email: guest?.email || "",
  phone: guest?.phone || "",
  documentType: guest?.documentType || "dni",
  documentNumber: guest?.documentNumber || "",
  nationality: guest?.nationality || "",
  direccion: guest?.direccion || "",
  localidad: guest?.localidad || "",
  codigoPostal: guest?.codigoPostal || "",
  fechaNacimiento: guest?.fechaNacimiento || "",
  sexo: guest?.sexo || "no_especifica",
  segment: guest?.segment || "LEISURE",
  cuilCuit: guest?.cuilCuit || "",
  companyId: guest?.companyId || null,
  agencyId: guest?.agencyId || null,
  vehiculoPatente: guest?.vehiculoPatente || "",
  vehiculoMarca: guest?.vehiculoMarca || "",
  vehiculoModelo: guest?.vehiculoModelo || "",
  vehiculoColor: guest?.vehiculoColor || "",
});
```

Adicionalmente verificar en el backend ( `PATCH /api/guests/:id` ) que use `{ ...req.body }` sin sobrescribir campos no enviados. El `updateGuest` en storage usa

`Partial<InsertGuest>` con `Drizzle .set()` , lo cual es correcto — solo actualiza los campos presentes. El problema está en el frontend enviando campos vacíos en lugar de omitirlos.

**Fix alternativo más robusto:** antes de hacer el PATCH, filtrar del objeto los campos que son string vacío y no corresponden a campos que el usuario quiso vaciar intencionalmente:

```
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  // Solo enviar campos que el usuario modificó o que tienen valor
  const payload = Object.fromEntries(
    Object.entries(formData).filter(([_, v]) => v !== "" && v !== undefined)
  );
  mutation.mutate(payload);
};
```

---

## SECCIÓN 4 — TARIFAS Y PAQUETES

### 4.1 Edición de tarifa no borra datos existentes

**Mismo patrón que clientes.** `RatePlanFormDialog` inicializa `formData` correctamente con los valores del plan, pero puede enviar campos vacíos en el PATCH.

**Verificar** que el `formData` inicial incluya todos los campos del schema de `rate_plans`:

```
const [formData, setFormData] = useState<Partial<InsertRatePlan>>>({
  name: ratePlan?.name || "",
  roomTypeId: ratePlan?.roomTypeId || "",
  baseRate: ratePlan?.baseRate || "",
  ratelpax: ratePlan?.ratelpax || null,
  rate2pax: ratePlan?.rate2pax || null,
  rate3pax: ratePlan?.rate3pax || null,
  rate4pax: ratePlan?.rate4pax || null,
  currency: ratePlan?.currency || "ARS",
  refundable: ratePlan?.refundable ?? "true",
  cancellationPolicy: ratePlan?.cancellationPolicy || "",
  // Agregar cualquier otro campo que esté en el schema pero falte aquí
});
```

Aplicar el mismo filtro de campos vacíos en el submit:

```
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  const payload = Object.fromEntries(
    Object.entries(formData).filter(([_, v]) => v !== "" && v !== undefined)
  );
  mutation.mutate(payload);
};
```

---

## 4.2 Tarifas por período de fechas

**Objetivo:** poder definir que una tarifa BAR tiene \$50.000/noche del 01/01 al 28/02 y \$65.000/noche del 01/03 al 30/11.

**Cambio en DB — agregar columnas a `rate_plans`:**

```
ALTER TABLE rate_plans
  ADD COLUMN valid_from date,
  ADD COLUMN valid_to date;
```

**Agregar en `shared/schema.ts`:**

```
validFrom: date("valid_from"),
validTo: date("valid_to"),
```

**Fix en `RatePlanFormDialog`: agregar dos campos de fecha opcionales:**

```
<div className="grid grid-cols-2 gap-4">
  <div className="grid gap-2">
    <Label>Válido desde (opcional)</Label>
    <Input
      type="date"
      value={formData.validFrom || ""}
      onChange={(e) => setFormData({ ...formData, validFrom: e.target.value || null
        data-testid="input-valid-from"
      />
    </div>
    <div className="grid gap-2">
      <Label>Válido hasta (opcional)</Label>
      <Input
        type="date"
        value={formData.validTo || ""}
        onChange={(e) => setFormData({ ...formData, validTo: e.target.value || null
```

```

        data-testid="input-valid-to"
      />
    </div>
  </div>

```

**Fix en la selección de tarifa al crear reserva:** cuando se muestran los planes aplicables, filtrar por vigencia:

```

const applicableRatePlans = ratePlans?.filter(rp => {
  if (rp.roomTypeId !== selectedRoomTypeId) return false;
  if (rp.validFrom && checkInDate < rp.validFrom) return false;
  if (rp.validTo && checkInDate > rp.validTo) return false;
  return true;
});

```

**En la lista de planes en `RatePlansPage` , mostrar el período de vigencia cuando esté definido:**

```

{plan.validFrom && plan.validTo && (
  <span className="text-xs text-muted-foreground">
    {formatDateAR(plan.validFrom)} → {formatDateAR(plan.validTo)}
  </span>
)}

```

## 4.3 Historial de tarifas — no sobrescribir, versionar

**Objetivo:** al editar una tarifa no perder el valor anterior. En lugar de hacer un PATCH que sobrescribe, crear una nueva versión y marcar la anterior como vencida.

**Implementación simplificada** (sin nueva tabla):

Al editar una tarifa existente, en lugar de `PATCH /api/rate-plans/:id` , el flujo debe ser:

1. Marcar la tarifa actual con `validTo = hoy - 1 día` (o la fecha que indique el operador)
2. Crear una nueva tarifa con los nuevos valores y `validFrom = hoy`

**Fix en `RatePlanFormDialog` cuando `isEditing === true` :**

Cambiar el botón de “Guardar cambios” por dos opciones:

- **“Actualizar directo”** → PATCH normal (para correcciones de datos, no de precio)

- **“Crear nueva versión”** → abre un campo de fecha “¿Desde cuándo aplica el nuevo precio?” y luego:

1. **PATCH a la tarifa actual** con `{ validTo: fechaElegida - 1 }`
2. **POST de nueva tarifa con los nuevos valores** y `{ validFrom: fechaElegida, name: plan.name }`

```
const createVersionMutation = useMutation({
  mutationFn: async ({ newData, effectiveFrom }: { newData: Partial<InsertRatePla
    // 1. Cerrar la versión actual
    await apiRequest("PATCH", `/api/rate-plans/${ratePlan!.id}`, {
      validTo: subDays(new Date(effectiveFrom), 1).toISOString().split("T")[0]
    });
    // 2. Crear nueva versión
    return apiRequest("POST", "/api/rate-plans", {
      ...newData,
      validFrom: effectiveFrom,
      validTo: null,
    });
  },
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["/api/rate-plans"] });
    toast({ title: "Nueva versión de tarifa creada" });
    onSuccess();
    onOpenChange(false);
  }
});
```

---

## NOTAS TÉCNICAS

- `bedTypeId` en el schema es `varchar/UUID` — nunca usar `parseInt()` ni `Number()` en ese campo.
- `reservation_changelog` ya existe con columnas: `reservationId`, `fecha`, `operador`, `tipo`, `descripcion`. Reutilizar para registrar check-ins retroactivos con `tipo: "checkin_retroactivo"`.
- `guests.companyId` ya existe en el schema — solo falta exponerlo en el formulario de edición.
- El filtro de `applicableRatePlans` por fecha aplica solo cuando `validFrom` / `validTo` tienen valor; planes sin período son “siempre vigentes”.

- **Al crear la migración de `valid_from / valid_to` en `rate_plans`, usar `ALTER TABLE` con `DEFAULT NULL` para no afectar registros existentes.**
- **No modificar la lógica de cálculo de balance en el folio — ya funciona correctamente con `totalRoomAmount + charges - payments`.**